

REACT RECORD – TYPED FORMAT

Typed from the uploaded handwritten React Record PDF. Wording and code have been kept as close to the source as readable.

Experiment – 1

Installation and Setup

Aim

To setup react for installation and display welcome message.

Source Code

Steps Step 1: Download node.js from <https://nodejs.org>

Step 2: Install node.js by following the installation. After installation complete, install Visual Studio Code (VS Code).

Step 3: Open VS Code and create a new project folder.

Step 4: Click File → Open Folder, create a new folder named My-First-app. Open it in VS Code.

Step 5: Open the terminal in VS Code by selecting Terminal → New Terminal.

Step 6: In the terminal, type the following command:

```
npm create vite@latest
```

If it shows an error named PowerShell block script execution, then put the command:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Step 7: When prompted, enter the project name:

```
Project Name: my-react-app
```

Step 8: Select the react framework using the arrow keys and press Enter. Then select javascript as the variant.

Step 9: Navigate the project folder and install the required packages:

```
cd my-react-app
```

```
npm install
```

Step 10: Start the development server by running:

```
npm run dev
```

Step 11: The Vite development server will start and display a local URL:

```
http://localhost:5173/
```

Step 12: Copy the URL and paste it in your web browser. The default React application page will be displayed.

```
App.jsxfunction App() {
  return (
    <h1>Hello React!</h1>
  );
}

export default App;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 2

Components

Aim

React application to create components.

Source Code

```
Header.jsxfunction Header() {
  return (
    <header>
      <h1>My Website</h1>
      <nav>
        <ul>
          <li>Apple</li>
          <li>Orange</li>
        </ul>
      </nav>
    </header>
  );
}

export default Header;
```

```
Footer.jsxfunction Footer() {
  return (
    <footer>
      <p>© 2026 My website. All rights reserved.</p>
    </footer>
  );
}

export default Footer;
```

```
App.jsximport Header from './Header.jsx';
import Footer from './Footer.jsx';
```

```
function App() {
  return (
    <>
      <Header />
      <Footer />
    </>
  );
}

export default App;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 3

Demonstration of Props

Aim

Write a program to demonstrate props.

Source Code

```
Student.jsxfunction Student(props) {
  return (
    <div>
      <h2>Name: {props.name}</h2>
      <p>Course: {props.course}</p>
    </div>
  );
}

export default Student;
```

```
App.jsximport Student from './Student';

function App() {
  return (
    <div>
      <Student name="Arun" course="Bsc. CS" />
      <Student name="Anu" course="BCA" />
    </div>
  );
}

export default App;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 4

Demonstration of Destructuring Props

Aim

Write a program to demonstrate destructure props.

Source Code

```
Student.jsxfunction Student({name, course}) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{course}</p>
    </div>
  );
}

export default Student;
```

```
App.jsximport Student from './Student';

function App() {
  return (
    <div>
      <Student name="Arun" course="Bsc. CS" />
      <Student name="Anu" course="BCA" />
    </div>
  );
}

export default App;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 5

Demonstration of useState

Aim

Write a program to demonstrate useState.

Source Code

```
Counter.jsximport { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
```

```

    return (
      <div>
        <h2>{count}</h2>
        <button onClick={() => setCount(count + 1)}>
          Increment
        </button>
      </div>
    );
  }
}

export default Counter;

```

App.jsx import Counter from './Counter';

```

function App() {
  return (
    <Counter />
  );
}

export default App;

```

Result

The above program is executed successfully and the output is verified.

Experiment – 6

Typescript program to print a message

Aim

Write a typescript program to print a message "Welcome to Mar Ivanios College" using arrow function.

Source Code

App.jsx

```

const App = () => {
  return (
    <header>
      <h1>Welcome to Mar Ivanios College</h1>
    </header>
  );
}

export default App;

```

Result

The above program is executed successfully and the output is verified.

Experiment – 7

Typescript demonstrating state using string

Aim

Write a typescript program using useState string.

Source Code

```
App.jsximport { useState } from 'react';

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
          setName(e.target.value)
        }
      />
      <h2>Hello {name}</h2>
    </div>
  );
}
```

Result

The above program is executed successfully and the output is verified.

Experiment – 8

Typescript demonstrating state using number

Aim

Write a typescript program using useState Number.

Source Code

```
App.jsximport { useState } from 'react';

function App() {
  const [count, setCount] = useState<number>(0);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

```
    );  
  }  
  
  export default App;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 9

Demonstration of Controlled Component

Aim

Write a program to demonstrate controlled component.

Source Code

```
App.jsximport React, { useState } from 'react';  
  
function ControlledComponent() {  
  const [name, setName] = useState("");  
  
  const handleChange = (event) => {  
    setName(event.target.value);  
  };  
  
  return (  
    <div>  
      <h2>Controlled Component</h2>  
      <label>Enter your Name:</label>  
      <input  
        type="text"  
        value={name}  
        onChange={handleChange}  
      />  
      <p>You entered: {name}</p>  
    </div>  
  );  
}  
  
export default ControlledComponent;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 10

Demonstration of Uncontrolled Component

Aim

Write a program to demonstrate Uncontrolled Component.

Source Code

```
App.jsximport React, { useRef } from 'react';

function UncontrolledComponent() {
  const nameRef = useRef();

  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Entered Name: " + nameRef.current.value);
  };

  return (
    <div>
      <h2>Uncontrolled Component</h2>
      <form onSubmit={handleSubmit}>
        <label>Name:</label>
        <input
          type="text"
          ref={nameRef}
          placeholder="Enter your name"
        />
        <br />
        <button type="submit">Submit</button>
      </form>
    </div>
  );
}

export default UncontrolledComponent;
```

Result

The above program is executed successfully and the output is verified.

Experiment – 11

To create panes in React

Aim

Write a program to create panes in React.

Source Code

```
App.jsxfunction App() {
  return (
    <div>
      <h1>Creating Panes in React</h1>
    </div>
  );
}
```

```

    <div
      style={{
        width: "45%",
        border: "1px solid black",
        padding: "20px",
        display: "inline-block"
      }}
    >
      <h2>Pane 1</h2>
      <p>This is the first Pane</p>
    </div>

    <div
      style={{
        width: "45%",
        border: "1px solid black",
        padding: "20px",
        display: "inline-block"
      }}
    >
      <h2>Pane 2</h2>
      <p>This is the second pane.</p>
    </div>
  </div>
);
}

export default App;

```

Result

The above program is executed successfully and the output is verified.

Experiment – 12

To create two panes using Flexbox

Aim

Write a program to create two Panes using Flexbox.

Source Code

```

App.jsxfunction App() {
  return (
    <div style={{display: "flex"}}>
      <div
        style={{
          flex: 1,
          border: "1px solid black",
          padding: "20px"
        }}
      >
        <h2>Left Pane</h2>

```

```

        <p>This is the left Pane</p>
      </div>

      <div
        style={{
          flex: 1,
          border: "1px solid black",
          padding: "20px"
        }}
      >
        <h2>Right Pane</h2>
        <p>This is the right pane.</p>
      </div>
    </div>
  );
}

export default App;

```

Result

The above program is executed successfully and the output is verified.

Experiment – 13

Demonstration of Simple Routing

Aim

Write a program to demonstrate Simple Routing.

Source Code

```

App.jsximport { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

```

```

        </Routes>
    </BrowserRouter>
  );
}

export default App;

```

Result

The above program is executed successfully and the output is verified.

Experiment – 14

To create an XML HTTP Request Object

Aim

Write a program to create an XML HTTP Request Object.

Source Code

```

index.html<!DOCTYPE html>
<html>
<body>
  <div id="demo">
    <h2>Let Ajax Change this text</h2>
    <button type="button" onclick="loadDoc()">Change Content</button>
  </div>

  <script src="loadDoc.js"></script>
</body>
</html>

```

```

loadDoc.jsfunction loadDoc() {
  const xhttp = new XMLHttpRequest();

  xhttp.onload = function() {
    document.getElementById("demo").innerHTML =
      this.responseText;
  };

  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}

```

ajax_info.txt Hello! This content is loaded using AJAX.

Result

The above program is executed successfully and the output is verified.

Experiment – 15

To fetch API for modern Ajax Request

Aim

Write a program to fetch API for modern Ajax Request.

Source Code

```
index.html<!DOCTYPE html>
<html>
<head>
  <title>Fetch API Example</title>
</head>
<body>
  <h2>Fetch User Data</h2>
  <button onclick="getUser()">Get User</button>
  <p id="result"></p>

  <script>
    function getUser() {
      fetch("https://jsonplaceholder.typicode.com/users/1")
        .then(response => response.json())
        .then(data => {
          document.getElementById("result").innerHTML =
            "Name: " + data.name + "<br>" +
            "Email: " + data.email;
        })
        .catch(error => {
          document.getElementById("result").innerHTML =
            "Error fetching data";
        });
    }
  </script>
</body>
</html>
```

Result

The above program is executed successfully and the output is verified.